The Journal of Open Source Software

FastVPINNs: An efficient tensor-based Python library for solving partial differential equations using hp-Variational Physics Informed Neural Networks

Thivin Anandh [1¶], **Divij Ghose** [1], **and Sashikumaar Ganesan** [1¶]

1 Department of Computational and Data Sciences, Indian Institute of Science, Bangalore, India ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Patrick Diehl

Reviewers:

- @pescap
- @ziyiyin97

Submitted: 17 May 2024
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Introduction

Partial differential equations (PDEs) are essential in modeling physical phenomena such as heat transfer, electromagnetics and fluid dynamics. The current progress in the field of scientific machine learning (SciML) has resulted in incorporating deep learning and data-driven methods to solve PDEs. Physics informed neural networks introduced by (Raissi et al., 2019) works by incorporating the governing equations of the physical systems into the Neural Network's loss function. Let us consider a two-dimensional Poisson equation as an example which reads as follows:

$$-\nabla^2 u(x) = f(x), \quad \Omega \in \mathbb{R}^2, \tag{1}$$
$$u(x) = g(x), \quad x \in \partial\Omega. \tag{2}$$

Here, $x \in \Omega$ is the spatial co-ordinate, $u(x)$ is the solution of the PDE, $f(x)$ is a known source term and $g(x)$ is the value of the solution on the domain boundary, $\partial\Omega$.

Physics-informed neural networks (PINNs), introduced by (Raissi et al., 2019), work by incorporating the governing equations of the physical systems and the boundary conditions as physics-based constrains to train the neural networks. The core idea of PINNs lies in the ability of obtaining the gradients of the solutions with respect to the input using the automatic differention routines available within deep learning frameworks such as tensorflow (Abadi et al., 2015). The loss function for the PINNs can be written as follows:

$$\begin{aligned} L_p(W,b) &= \frac{1}{N_T}\sum_{t=1}^{N_T} \left|(-\Delta u_{\text{NN}}(x_t;W,b) - f(x_t))\right|^2, \\ L_b(W,b) &= \frac{1}{N_D}\sum_{d=1}^{N_D} \left|u_{\text{NN}}(x_d;W,b) - g(x_d)\right|^2, \\ L_{\text{PINN}}(W,b) &= L_p + \tau L_b. \end{aligned}$$

Here, $u_{\text{NN}}(x;W,b)$ is the approximate solution of the PDE obtained from neural network with weights W and biases b. In addition, N_T is the total number of training points in the interior of the domain Ω, N_D is the total number of training points on the boundary $\partial\Omega$, τ is a scaling factor applied to control the penalty on the boundary term and $L_{\text{PINN}}(W,b)$ is the loss function of the PINNs.

Variational Physics informed neural networks (VPINNs) (Kharazmi et al., 2019) are an extension of PINNs, where the weak form of the PDE is used in the loss function of the neural network. The weak form of the PDE is obtained by multiplying the PDE with a test function, integrating over the domain and applying integration by parts to the higher order derivative terms. The method of hp-VPINNs (Kharazmi et al., 2021) was subsequently developed to increase the accuracy using h-refinement (increasing number of elements) and p-refinement (increasing the number of test functions).The loss function of hp-VPINNs with `N_elem` elements in the domain can be written as follows

$$
\begin{aligned}
L_v(W,b) &= \frac{1}{\texttt{N_elem}} \sum_{k=1}^{\texttt{N_elem}} \left| \int_{K_k} \left(\nabla u_{\text{NN}}(x;W,b) \cdot \nabla v_k \; - \; f v_k \right) dK \right|^2, \\
L_b(W,b) &= \frac{1}{N_D} \sum_{d=1}^{N_D} \left| u_{\text{NN}}(x;W,b) - g(x) \right|^2, \\
L_{\text{VPINN}}(W,b) &= L_v + \tau L_b.
\end{aligned}
$$

Where K_k is the kth element in the domain, v_k is the test function in the respective element. Further, $L_v(W,b)$ is the weak form PDE residual and $L_{\text{VPINN}}(W,b)$ is the loss function of the hp-VPINNs. For more information on the derivation of the weak form of the PDE and the loss function of hp-VPINNs, refer to (Anandh et al., 2024) and (Ganesan & Tobiska, 2017)

Statement of need

The existing implementation of hp-VPINNs framework (Kharazmi, 2023) suffers from two major challenges. One being the inabilty of the framework to handle complex geometries and the other being the increased training time associated with the increase in number of elements within the domain. In the work (Anandh et al., 2024), we presented FastVPINNs, which addresses both of these challenges. FastVPINNs handles complex geometries by using bilinear transformation, and it uses a tensor-based loss computation to reduce the dependency of training time on number of elements. The current implementation of FastVPINNs can acheive an speed-up of upto a 100 times when compared with the existing implementation of hp-VPINNs. We have also shown that with proper hyperparameter selection, FastVPINNs can outperform PINNs both in terms of accuracy and training time, especially for problems with high frequency solutions.

Our FastVPINNs framework is built using TensorFlow 2 (Abadi et al., 2015), and provides an elegant API for users to solve both forward and inverse problems for PDEs like the Poisson, Helmholtz and Convection-Diffusion equations.The framework is well-documented with examples, which can enable users to get started with the framework with ease. The framework also provides an API to all the modules for users to write their custom functionalities. With the current level of API abstraction, users should be able to solve PDEs with less than five API calls as shown in the minimal working example section.

The ability of the framework to allow users to train a hp-VPINNs to solve a PDE both faster and with minimal code, can result in widespread application of this method on several real-world problems, which often require complex geometries with a large number of elements within the domain.

Modules

The FastVPINNs framework consists of five core modules, which are Geometry, FE, Data, Physics and Model.

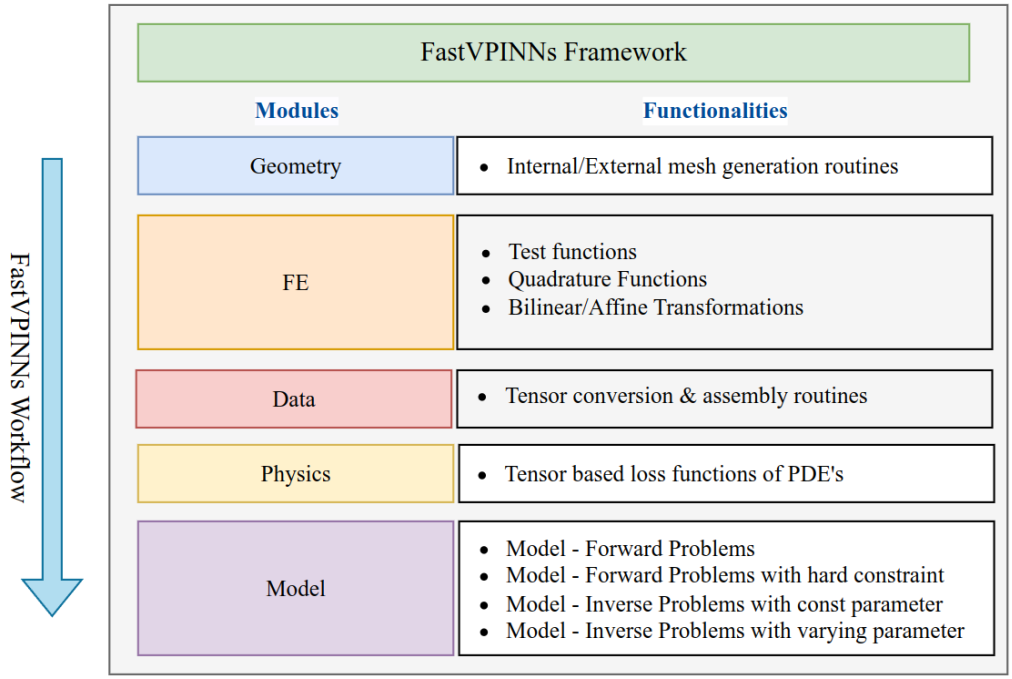

Figure 1: FastVPINNs Modules

Geometry Module

This module provides the functionality to define the geometry of the domain. The user can either generate a quadrilateral mesh internally or read an external `.mesh` file. The module also provides the functionality to obtain boundary points for complex geometries.

FE Module

The FE module is responsible for handling the Finite Element test functions and their gradients. The module's functionality can be broadly classified into into four categories:

- **Test functions**: Provides the test function values and its gradients for a given reference element.
- **Quadrature functions**: Provides the quadrature points and weights for a given element based on the quadrature order selected by the user. These values will be used for numerical integration of the weak form of the PDE.
- **Transformation functions**: Provides the implementation of transformation routines such as 'Affine' transformation and 'Bilinear' transformation. This can be used to transform the test function values and gradients from the reference element to the actual element.
- **Finite Element Setup**: Provides the functionality to set up the test functions, quadrature rules and the transformation for every element and save them in a common wrapper class to access them. Further, it also hosts functions to plot the mesh with quadrature points, assign boundary values based on the boundary points obtained from the geometry module and calculation of the forcing term.

Remark: The module is named FE (Finite Element) Module because of its similarities with classical FEM routines, such as test functions, numerical quadratures and transformations. However, we would like to reiterate that our framework is not an FEM solver, but a hp-VPINNs solver

DRAFT

Data Module:

The Data module collects data from all modules which are required for training and converts them to a tensor data type with required precision. It also assembles the test function values and gradients to form a three-dimensional tensor, which will be used during the loss computation.

Physics Module:

This module contains functions which are used to compute the variational loss function for different PDEs. These functions accept the test function tensors and the predicted gradients from the neural network along with the forcing matrix to compute the PDE residuals.

Model Module:

This module contains custom subclasses of the `tensorflow.keras.Model` class, which are used to train the neural network. The module provides the functionality to train the neural network using the tensor based loss computation and also provides the functionality to predict the solution of the PDE for a given input.

Minimal Working Example

With the higher level of abstraction provided by the FastVPINNs framework, users can solve a PDE with just five API calls. Shown below is a minimal working example to solve the Poisson equation using the FastVPINNs framework.

```
#load the geometry
domain = Geometry_2D("quadrilateral", "internal",
                     100, 100, "./")
cells, boundary_points =
                domain.generate_quad_mesh_internal(\
                x_limits=[0, 1], y_limits=[0, 1],\
                n_cells_x=4, n_cells_y=4,\
                num_boundary_points=400)

# Note: Users need to provide the necessary
#       boundary values and the forcing function

# load the FEspace
fespace = Fespace2D(domain.mesh,cells,boundary_points,\
    domain.mesh_type,fe_order=5,\
    fe_type="jacobi",quad_order=5,\
    quad_type="legendre", \
    fe_transformation_type="bilinear"\
    bound_function_dict=bound_function_dict,\
    bound_condition_dict=bound_condition_dict,\
    forcing_function=rhs,\
    output_path=i_output_path,\
    generate_mesh_plot=True)

# Instantiate Data handler
dh = datahandler2D(fespace, domain, dtype=tf.float32)

# Instantiate the model with the loss function for the model
model = DenseModel(layer_dims=[2, 30, 30, 30, 1],\
            learning_rate_dict=0.01,params_dict=params_dict,
```

DRAFT

```
            ## Loss function of poisson2D
            loss_function=pde_loss_poisson,
            input_tensors_list=\
                [in_tensor,
                dir_in,
                dir_out],
            orig_factor_matrices=\
                [dh.shape_val_mat_list,
                dh.grad_x_mat_list,
                dh.grad_y_mat_list],
            force_function_list=dh.forcing_function_list,\
            tensor_dtype=tf.float32,
            use_attention=i_use_attention,
            activation=i_activation,
            hessian=False)

# Train the model
for epoch in range(1000):
    model.train_step()
```

Testing

The FastVPINNs framework has a strong testing suite, which tests the core functionalities of the framework. The testing suite is built using the `pytest` framework and is integrated with the continuous integration pipeline provided by Github Actions. The testing functionalites can be broadly classified into the three categories:

- **Unit Testing**: Covers the testing of individual modules and their functionalities.
- **Integration Testing**: Covers the overall flow of the framework. Different PDEs are solved with different parameters to check if the accuracy after training is within the acceptable limits. This ensures that the collection of modules work together as expected.
- **Compatibility Testing**: The framework is tested with different versions of python such as 3.8, 3.9, 3.10, 3.11 and on different versions of OS such as Ubuntu, MacOS and Windows to ensure the compatibility of the framework across different platforms.

Acknowledgements

We thank Shell Research, India, for providing partial funding for this project. We are thankful to the MHRD Grant No.STARS-1/388 (SPADE) for partial support.

References

Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., … Zheng, X. (2015). *TensorFlow: Large-scale machine learning on heterogeneous systems*. https://doi.org/10.48550/arXiv.1603.04467

Anandh, T., Ghose, D., Jain, H., & Ganesan, S. (2024). *FastVPINNs: Tensor-driven acceleration of VPINNs for complex geometries*. https://doi.org/10.48550/arXiv.2404.12063

Ganesan, S., & Tobiska, L. (2017). *Finite elements: Theory and algorithms*. Cambridge University Press. https://doi.org/10.1017/9781108235013

DRAFT

Kharazmi, E. (2023). *hp-VPINNs: High-performance variational physics-informed neural networks*. https://github.com/ehsankharazmi/hp-VPINNs

Kharazmi, E., Zhang, Z., & Karniadakis, G. E. (2019). Variational physics-informed neural networks for solving partial differential equations. *arXiv Preprint arXiv:1912.00873*. https://doi.org/10.48550/arXiv.1912.00873

Kharazmi, E., Zhang, Z., & Karniadakis, G. E. (2021). hp-VPINNs: Variational physics-informed neural networks with domain decomposition. *Computer Methods in Applied Mechanics and Engineering*, *374*, 113547. https://doi.org/10.1016/j.cma.2020.113547

Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, *378*, 686–707. https://doi.org/10.1016/j.jcp.2018.10.045

DRAFT